

Local LLM Policy Gap Analyzer

USP & MVP Analysis

Fully Offline / No RAG / No External APIs / No Cloud Storage / 27+ Prompt Injection Mitigations

Team: T-reXploit | HACK-IITK 2025 (C3iHub) | April 2026

Unique Selling Proposition

The Whole Point Is That Nothing Leaves Your Machine

Most "privacy-first AI" tools still make network calls somewhere: telemetry, license validation, model inference. This one genuinely doesn't. After you pull out the Gemma3 model once over the internet, you can cut the cable, and it still works exactly the same. The Flask server, the LLM, the NIST reference files, the output PDFs, all of it runs on your hardware, on your LAN.

That's not just a nice feature. Think about what these documents actually contain. A network segmentation policy. A data breach response procedure. An access control framework. These are the exact documents an attacker would love to read before targeting your organization. Sending them to any cloud endpoint, even a reputable one, is a bad idea. This tool makes that bad idea structurally impossible because there's no endpoint to send them to.

This is also why there's no RAG pipelines and no external database. Both of those would require a server somewhere. The NIST CSF reference material is just static files sitting in `data/reference/` as part of the repo. It loads from the disk. That's it. No retrieval, no vector database, no moving parts that could be tampered with or exfiltrated.

The 27+ Prompt Injection Mitigations Are the Real Work Here

This is the part that actually took thought and effort, and it's the part that makes this more than a hackathon toy.

When your input isn't a typed chat message but an uploaded file (a PDF, a Word doc, a text file) the attack surface is completely different, and most people building LLM tools ignore it entirely. The content of that file gets parsed, extracted, and injected into a prompt. If you don't control what comes out of that file, someone can craft a document that hijacks the model.

Here is what that looks like in practice:

- **Unicode zero-width character injection:** Characters like U+200B, U+FEFF, U+200C are invisible on screen but visible to the tokenizer. They can split words across token boundaries to dodge content filters or carry invisible instructions the model treats as part of the system prompt. The codebase explicitly sanitizes these.
- **Fake NIST CSF section injection:** Someone embeds well-formatted fake NIST headers inside the policy body itself, trying to confuse the model about which document is the reference framework, and which is the input. If your prompt is just string concatenation, this works. The structural prompt templates in this pipeline make it significantly harder.
- **Context overflow via oversized document:** Submitting a 200KB document specifically to push the NIST framework reference out of the model's context window. The model then analyzes the policy with no standard to compare against and produces nonsense that looks like a real analysis. The 50KB policy size cap and 100KB prompt size cap are direct mitigations.
- **Non-visible PDF layer attacks:** A PDF can have content in annotation objects or streams that PyPDF2 extracts, but that doesn't appear in the rendered view. A human reviewer signs off on the document because it looks clean. The LLM gets a different version.
- **Metadata channel attacks:** Word documents have metadata fields (author, comments, tracked changes) that get extracted separately from body text. A payload can sit in a comment or track change annotation while the body looks clean.
- **Encoding-based bypass:** Submitting text in alternate Unicode normalizations (NFC vs NFD vs NFKC) or using homoglyphs to construct trigger phrases that pass string-based sanitization but are rendered identically by the tokenizer.
- **Template injection through structured fields:** If the policy contains strings like {system prompt} that the pipeline uses in string formatting, these can be resolved into unintended content.
- **Prompt delimiter confusion:** Embedding the pipeline's own structural delimiters inside the document body to break the integrity of the prompt construction.

Getting to 27+ variants means they sat down and enumerated the attack classes systematically: direct injection, encoding-based bypass, delimiter confusion, metadata channel attacks, template injection, size-based context overflow, nested instruction attacks, and more. That is not something you stumble into. Someone spent time thinking adversarial about this specific pipeline.

The reason for this matter is practical. A gap analysis tool that can be manipulated to output "fully compliant" for a non-compliant policy is not just useless; it is actively dangerous. It gives a false audit trail. The injection of hardening is what makes the output trustworthy enough to actually act on.

No RAG Means the Analysis Is Reproducible and the Framework Can't Be Poisoned

In a RAG-based system, the NIST framework would live in a vector database and get retrieved dynamically at query time. That creates a poisoning problem. If someone can modify what's in the database, they can make the model compare policies against a corrupted version of the standard. A non-compliant policy gets flagged as compliant. Nobody notices.

Because the framework here is loaded from static files checked into the repository, the comparison baseline is fixed and auditable. You can look at data/reference/ and see exactly what standards the model is analyzing against. You can version-control it. You can verify that it hasn't been touched. Run the same policy through the same version of the tool twice, and you'll get structurally consistent results, because nothing about the retrieval of step changes between runs.

That reproducibility is important for compliance use cases specifically. If you're using this as part of an audit trail, you need to be able to explain and replicate your methodology. "We uploaded the document to an AI tool" is not an explanation. "We ran version 1.1 of this tool against NIST CSF CIS MS-ISAC 2024 reference files, here's the output" is.

The LAN Server Is a Smarter Deployment Model Than It Looks

Getting new software onto every machine in an organization is painful. IT has procurement processes. Security has approval of workflows. End users have no admin rights. This can take months for a tool anyone could install in 10 minutes on a personal machine.

The LAN server model sidesteps all of that. One person whoever is running this starts the Flask server on their machine. They share the local IP and port in a Slack message. Everyone else opens a browser and uses it. No installation on anyone else's machine. No IT ticket. No software approval. The analyst's laptop is effectively the server for the team.

The rate limiting (2 jobs per IP, 10 total in queue) and the per-IP controls aren't just about preventing server overload. They also limit what a malicious machine on the LAN can do. Someone can't flood the queue from a compromised workstation and lock out legitimate users. The job history page showing all submitted jobs across all LAN clients creates natural accountability: everyone can see what was submitted and when.

The Security Headers and Input Hardening Are Load bearing, Not Boilerplate

The server implements X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and X-XSS-Protection: 1; mode=block. In the context of a tool that lets users upload documents and renders results in a browser, these matter concretely.

- **X-Content-Type-Options: nosniff** prevents a scenario where a maliciously uploaded file is served back with a MIME type the browser would execute.
- **X-Frame-Options: DENY** prevents clickjacking against the upload form, relevant if the LAN includes compromised machines being used to trick users into submitting documents to a different endpoint.

- **Path traversal protection** on file downloads validates that resolved paths stay within the output directory, preventing directory traversal attacks on the download endpoint.

These are not cargo-culted. In combination, they form a coherent web security posture for a locally hosted file handling application. Each header corresponds to a real attack scenario in this specific deployment context.

Minimum Viable Product

What It Actually Does, Without the Hype

You upload a policy document. The system reads it, runs it through a four-stage pipeline (gap analysis, policy revision, implementation roadmap, executive summary) using a local Gemma3:4b model with the NIST CSF framework as the reference. About 6 minutes later you can download 10 files: 5 text reports and 5 PDFs formatted well enough to hand to a manager or attach to an audit submission.

The PDF quality improvement is documented in BEFORE_AFTER_COMPARISON.md and it's not cosmetic. The before state was plain text output with equals-sign dividers. The after state is properly formatted with PDFs with styled headings, justified text, formatted bullet points, and a layout that looks like something a professional produced. That matters because the output goes to people: executives, auditors, compliance officers who judge credibility partly by presentation.

The live progress log on the status page (stage 1 of 6 through stage 6 of 6, updating via AJAX without a page reload) is a small detail that significantly changes the experience. Without it, users stare at a spinner for 6 minutes wondering if something broke. With it, they can see the pipeline progressing and have confidence that the tool is working.

The Defense Stack Before Anything Reaches the Model

The security layer isn't bolted on; it runs before the LLM sees a single character of the uploaded document:

- File type validation against an allowlist (.txt, .pdf, .docx only).
- Size check at upload (50MB limit) and again after extraction (50KB policy limit, 100KB prompt limit).
- Unicode sanitization stripping zero-width and invisible formatting characters before content reaches the prompt.
- Prompt structural templates with clearly delimited content positions rather than naive string concatenation.
- Model allowlisting so users cannot specify an arbitrary model name and force use of an unvetted model.
- Serialized job execution, so concurrent submissions cannot interfere with each other's prompt contexts.
- Path traversal protection on file downloads, validating that resolved paths stay within the output directory.

Each of those corresponds to a real attack vector against this specific type of system. None of them are there by default in a standard Flask app or LLM pipeline.

The rate_limiter.py Module Is the Most Underrated Signal of Maturity

The `rate_limiter.py` file is 380 lines of thread-safe queue management, persistent job history, per-IP limits, TTL cleanup, and file validation. A 380-line rate limiter is not something you write in a hackathon rush to impress judges. It's something you write because you've thought seriously about what happens when multiple real users hit the system simultaneously.

The persistent `data/job_history.json` that survives server restarts, the automatic cleanup when output files are deleted, and the JSON status API for in-page polling without full-page refreshes are similarly production-thinking features. These don't make a flashy demo, but they make the difference between a tool people use once and a tool that becomes part of a team's actual workflow.

Where the Honest Ceiling Is

Gemma3:4b is a 4-billion parameter model. For the specific task of reading a 40KB policy document, comparing it to a structured compliance framework, identifying concrete gaps, writing a revised version addressing those gaps, producing a phased roadmap, and summarizing it for executives, that is a lot to ask of a small model. The results are useful as a starting point and genuinely accelerate the work, but they need a human to verify before anyone acts on them. The README says this directly: "human review recommended." That's honest and it's the right framing.

The tool also has no memory across analyses. It cannot compare last quarter's policy to this quarter's and tell you what improved. It cannot track remediation progress over time. Each submission is completely stateless.

English only. No ISO 27001, GDPR, PCI-DSS, or HIPAA support. NIST CSF is the only framework in the reference files currently.

None of those gaps invalidate what it does well. For an organization that currently has no gap analysis process at all (no consultant, no GRC tool, no internal compliance team) this produces a real, usable analysis artifact in 6 minutes on hardware they already own, without their most sensitive documents touching a server they don't control. That is the actual value proposition, stated plainly.